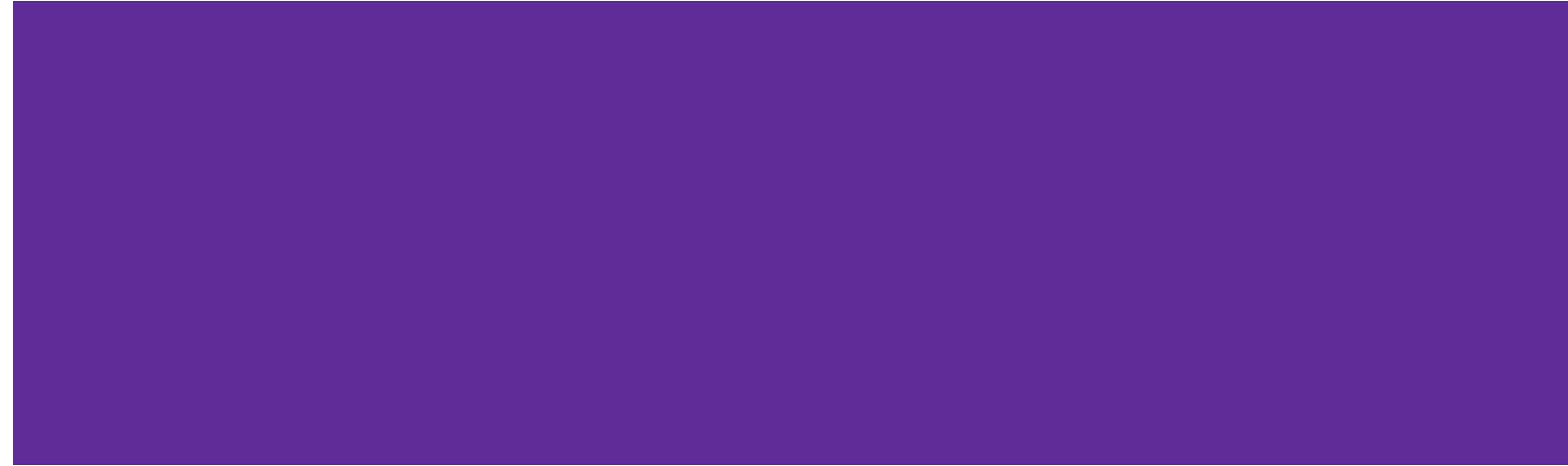
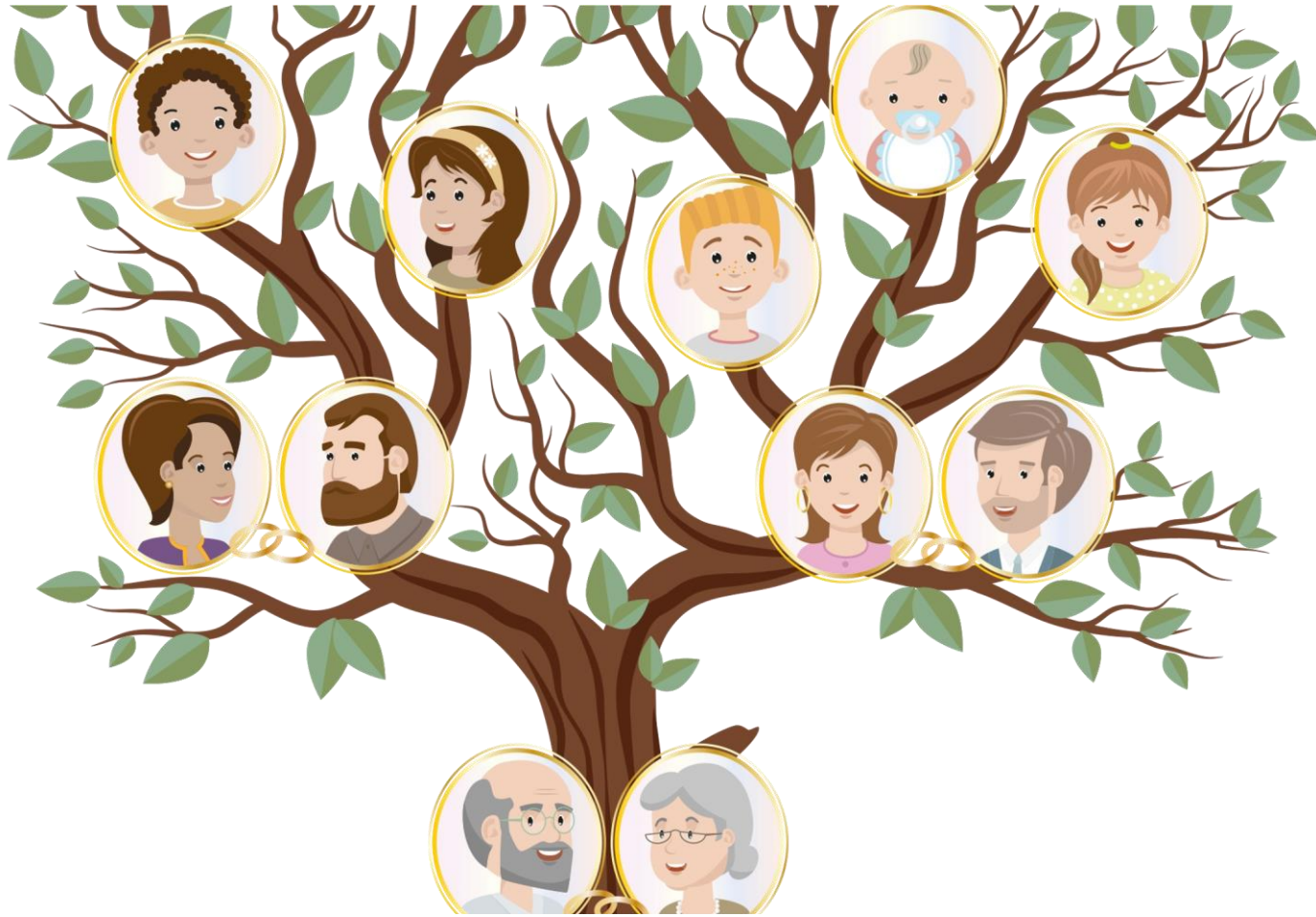


Tree

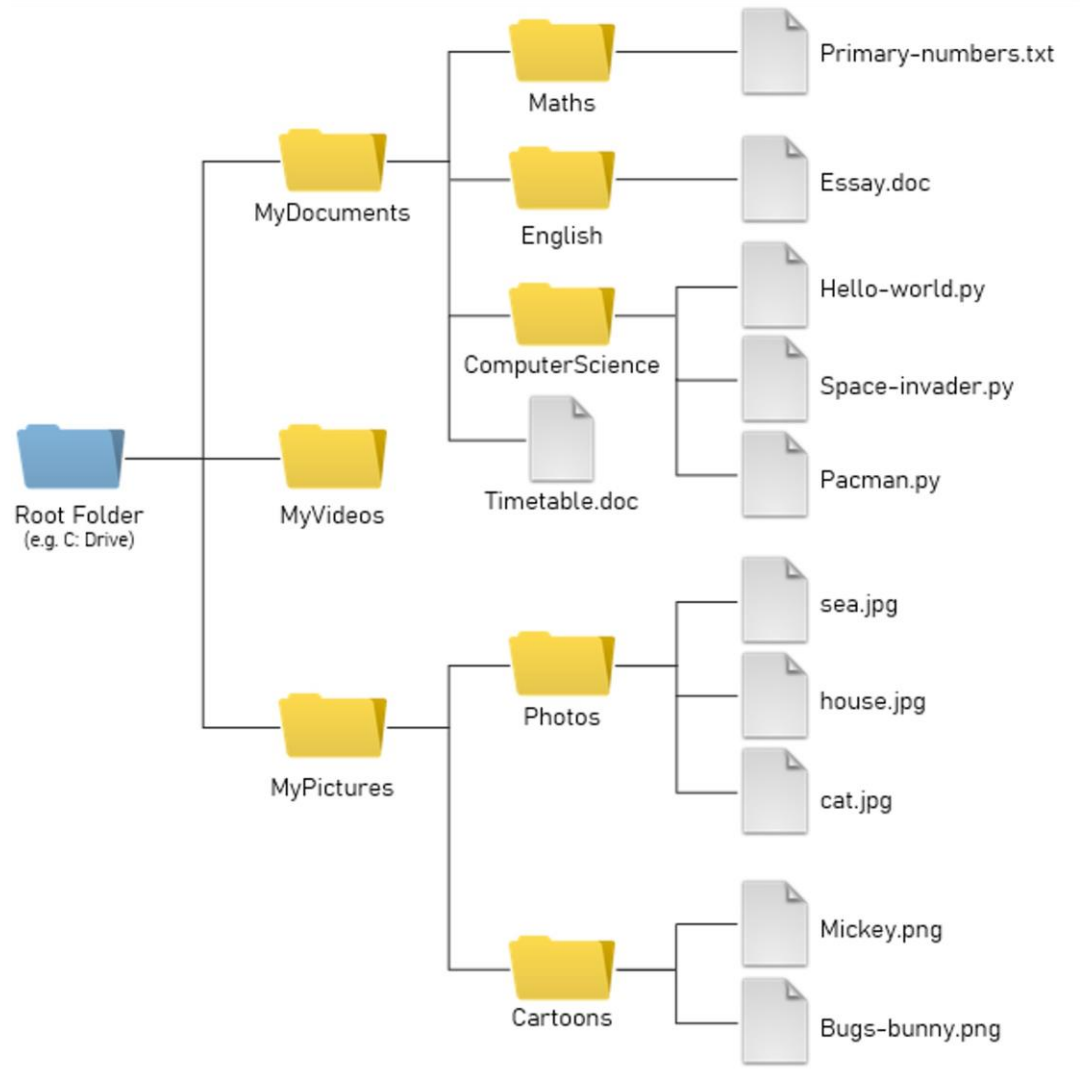
binh.nguyenthanh@phenikaa-uni.edu.vn

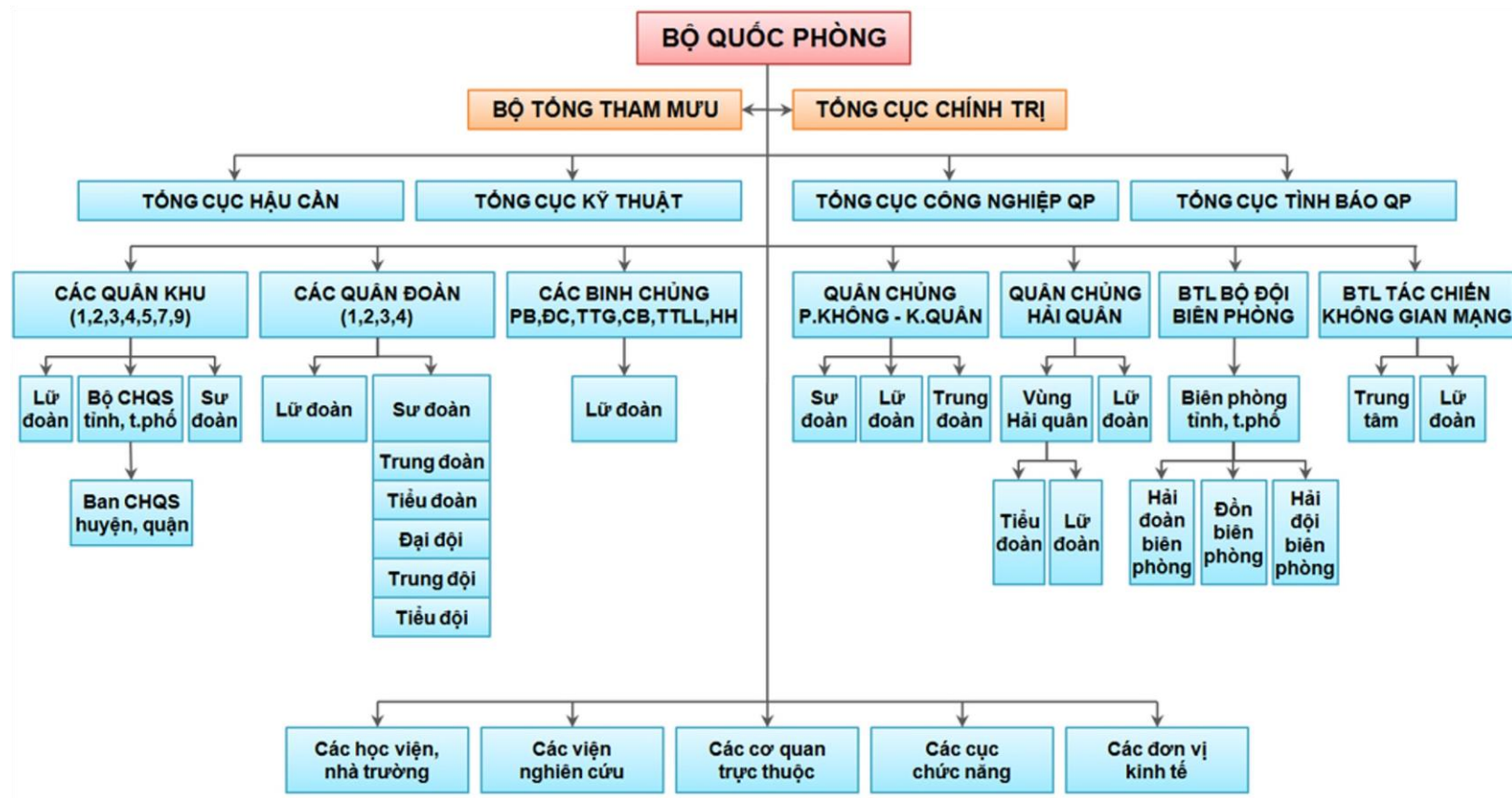




Cây gia phả

Cây thư mục

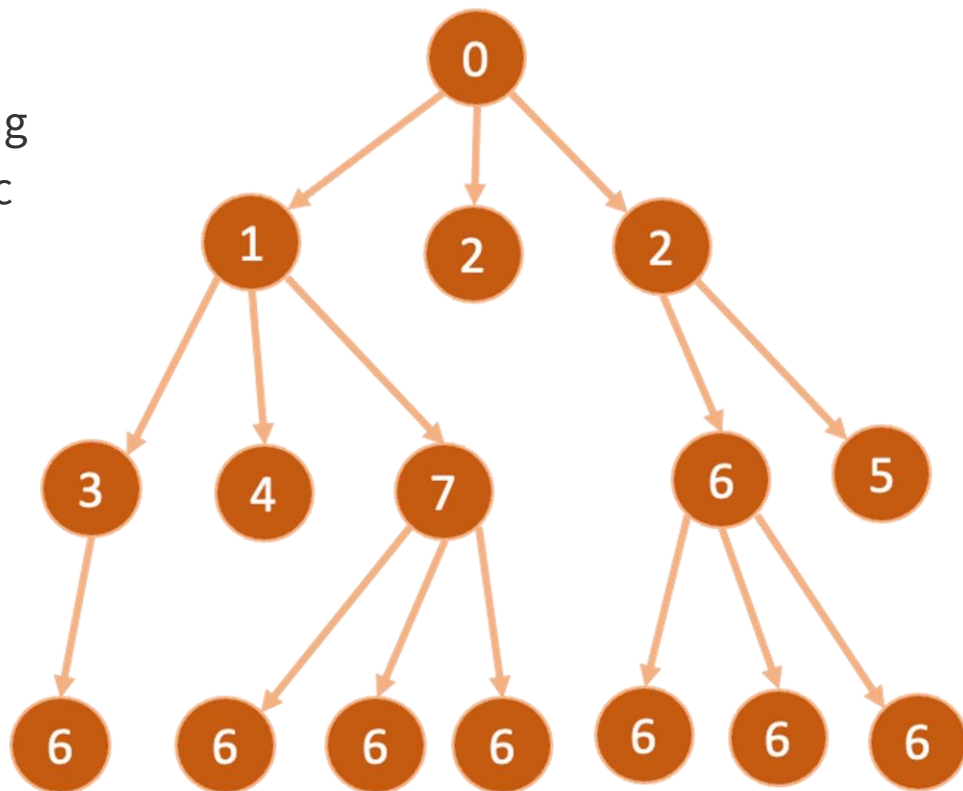


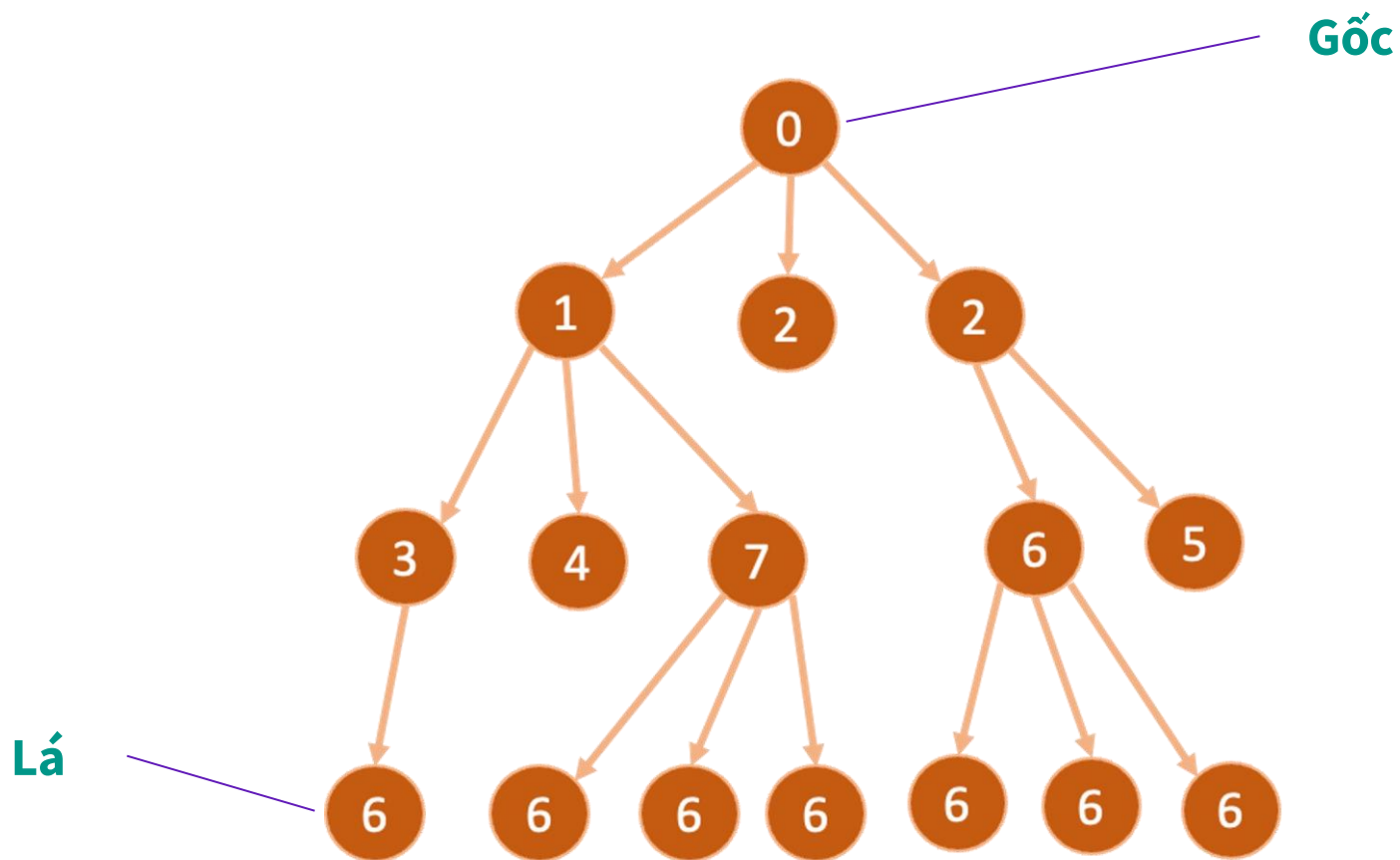


Cây tổ chức

Khái niệm

Cây là một cấu trúc dữ liệu được sử dụng rộng rãi gồm một tập hợp các nút, được liên kết với nhau theo quan hệ cha-con

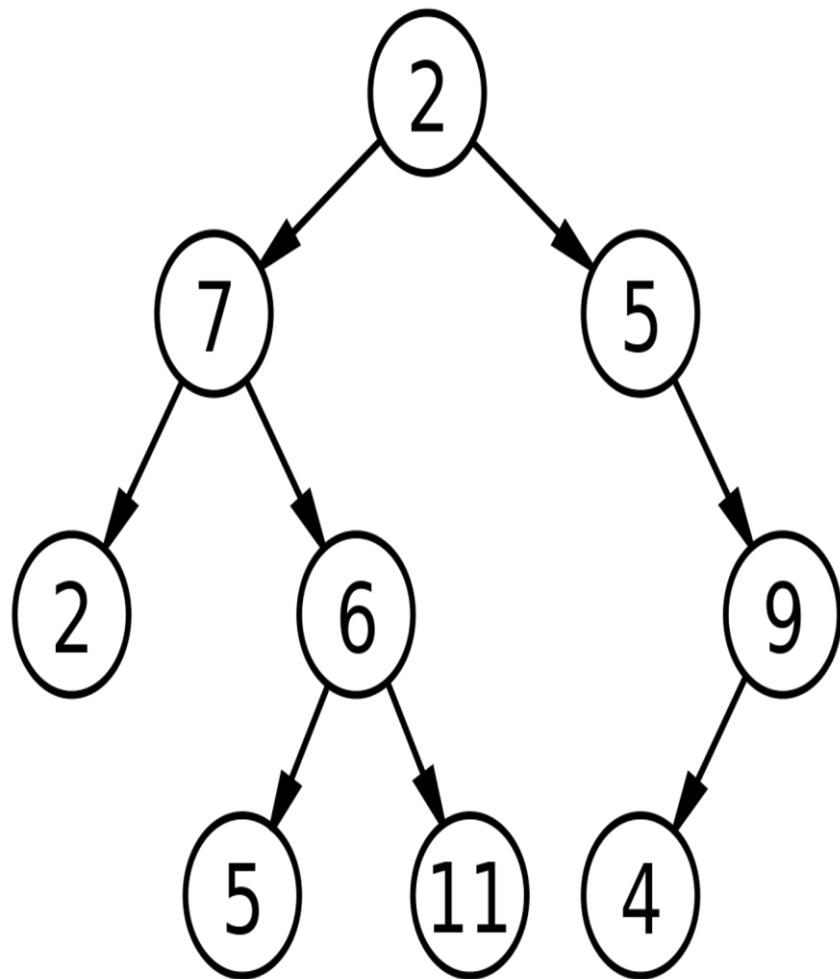




Cây nhị phân

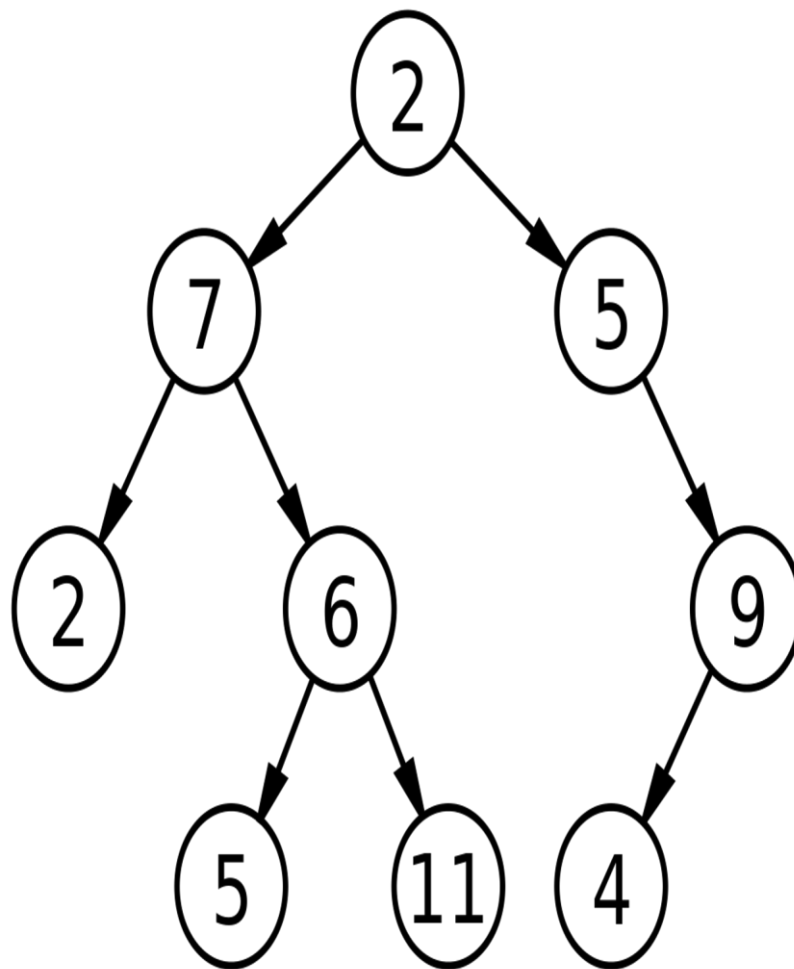
Cây nhị phân là cây mà mọi nút trên cây chỉ có tối đa hai nút con.

Một nút con bên trái và một nút con phải



Lưu trữ bằng con trỏ

```
class node {  
    node* left;  
    int value;  
    node* right;  
};
```



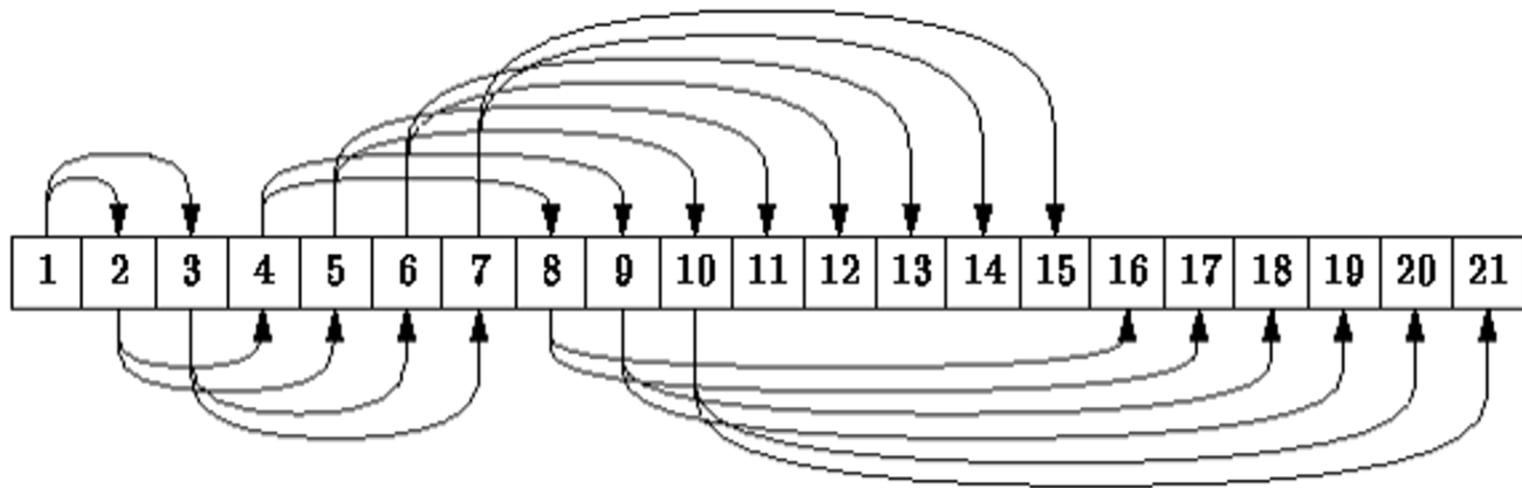
Lưu trữ bằng mảng

Nút cha ở vị trí n thì có:

- Nút con trái ở $2 * n$
- Nút con phải ở $2 * n + 1$

Nút con ở vị trí n thì có:

- Nút cha ở $[n/2]$



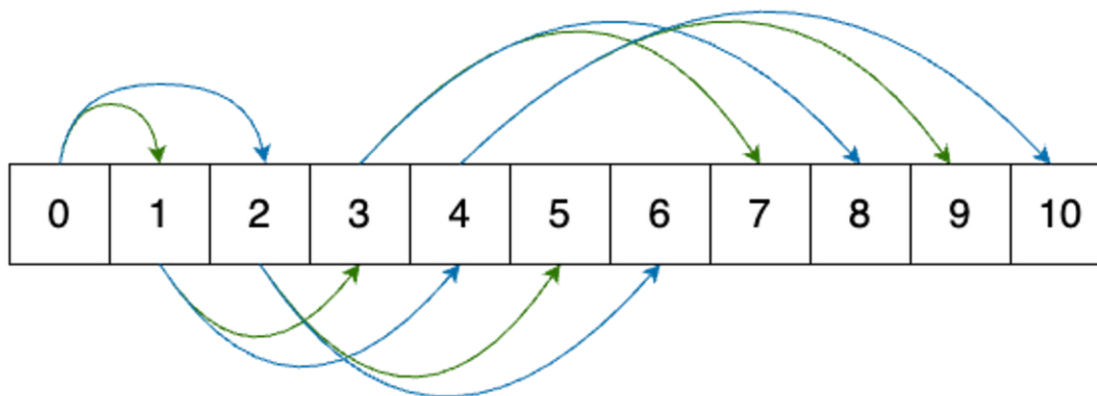
Lưu trữ bằng mảng

Nút cha ở vị trí n thì có:

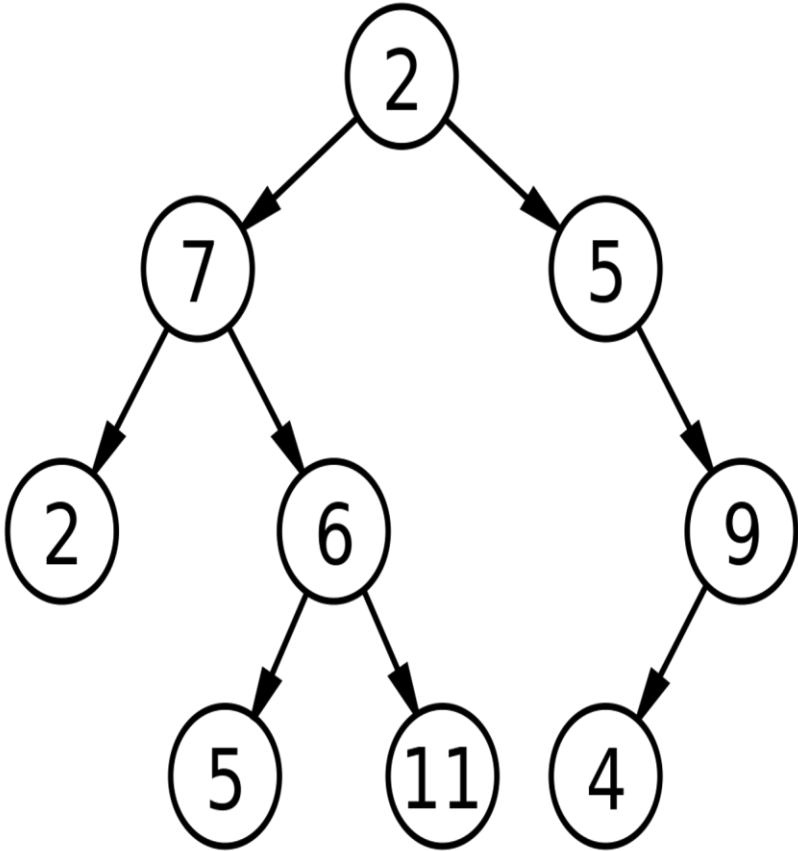
- Nút con trái ở $2 * n + 1$
- Nút con phải ở $2 * n + 2$

Nút con ở vị trí n thì có:

- Nút cha ở $[(n - 1)/2]$



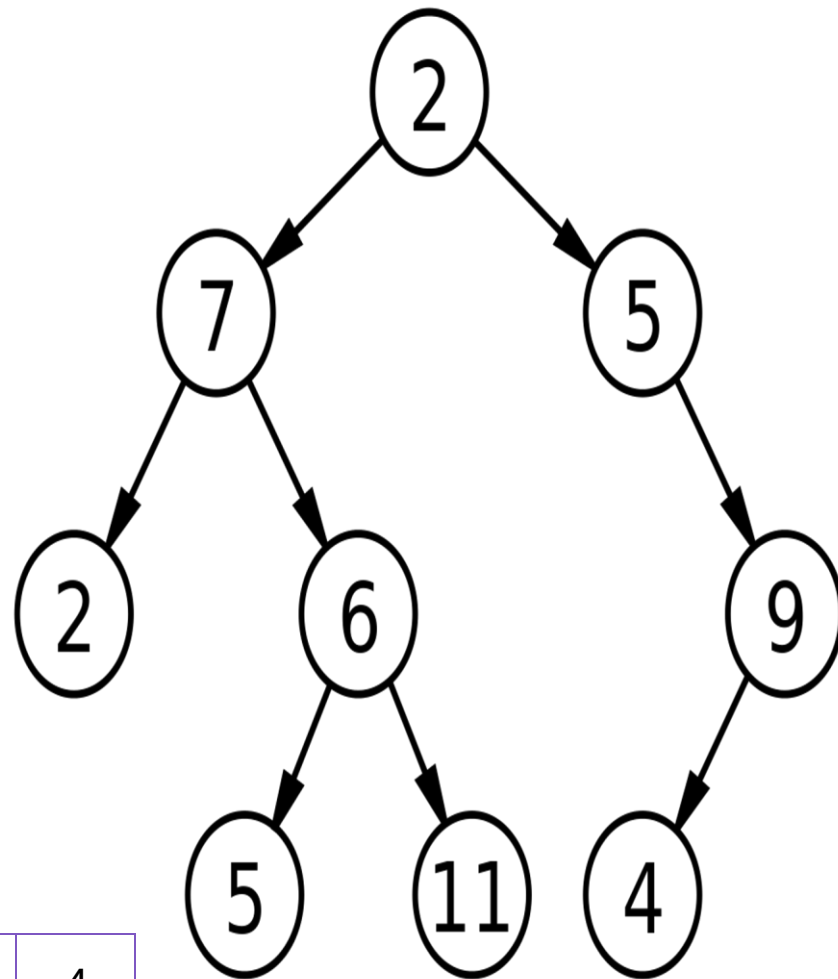
Lưu trữ bằng mảng



2	7	5	2	6		9			5	11			4
0	1	2	3	4	5	6	7	8	9	10	11	12	13

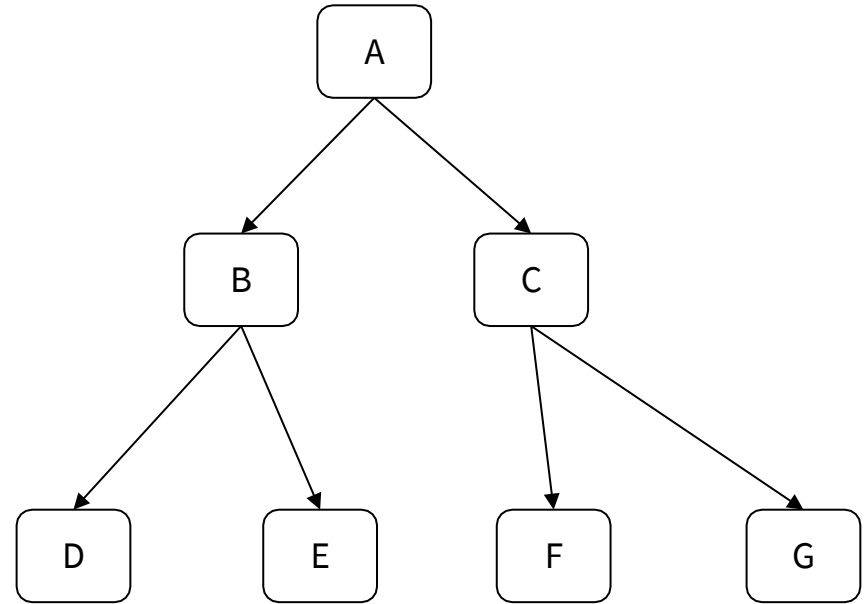
Duyệt tiền thứ tự

- Nếu Cây là rỗng Return
- Thăm A
- Duyệt tiền thứ tự cây con gốc trái
- Duyệt tiền thứ tự cây con gốc phải



2	7	2	6	5	11	5	9	4
---	---	---	---	---	----	---	---	---

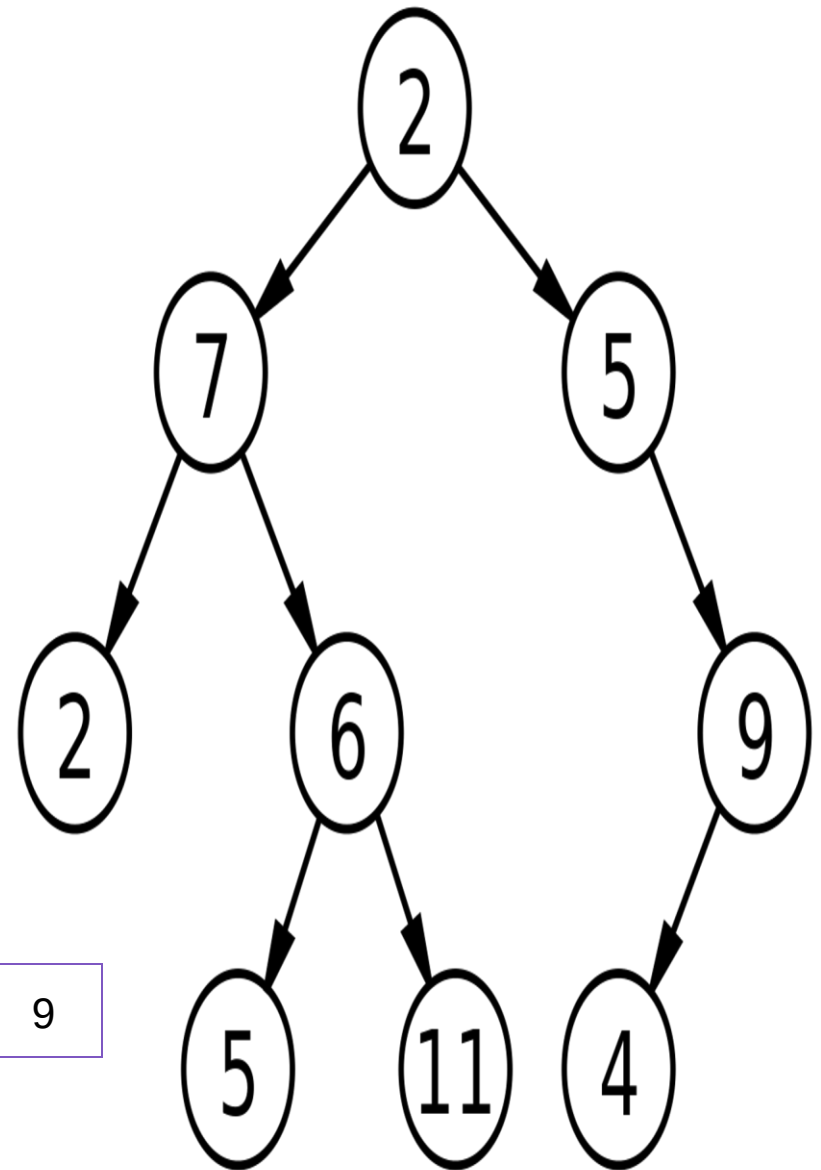
Duyệt tiền thứ tự



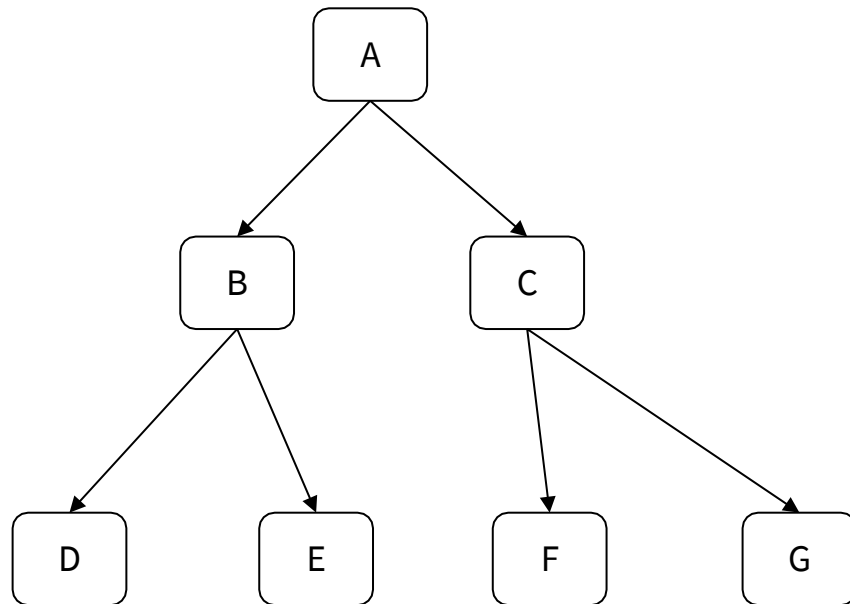
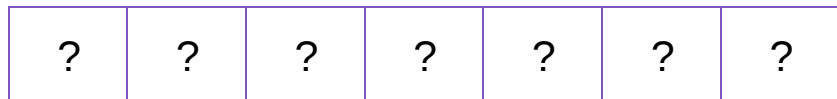
Duyệt trung thứ tự

- Nếu Cây là rỗng Return
- Duyệt trung thứ tự cây con gốc trái
- Thăm A
- Duyệt trung thứ tự cây con gốc phải

2	7	5	6	11	2	5	4	9
---	---	---	---	----	---	---	---	---



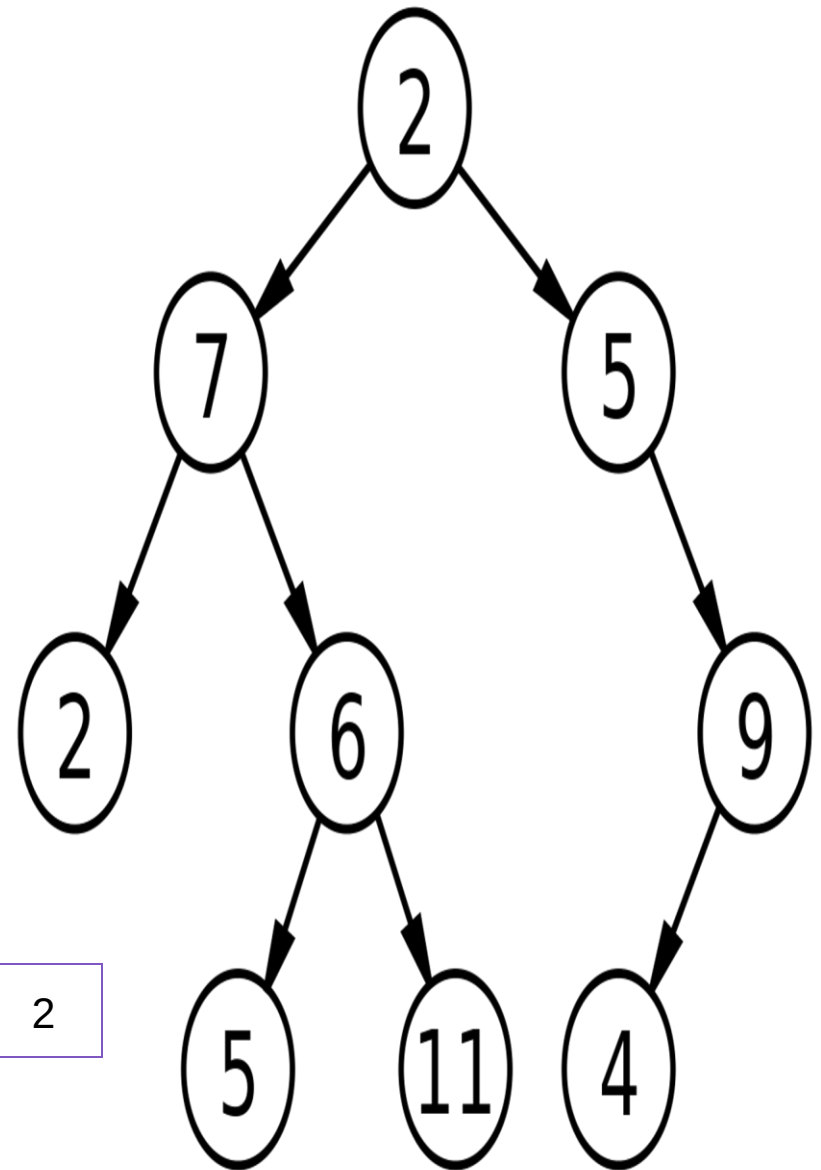
Duyệt trung thứ tự



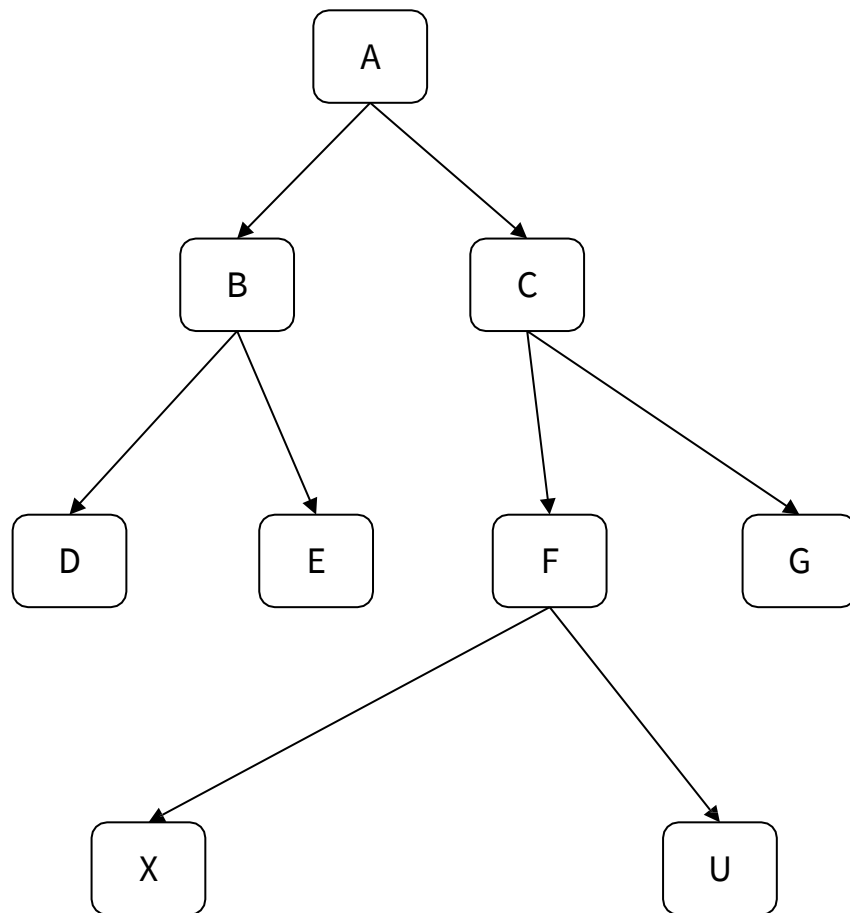
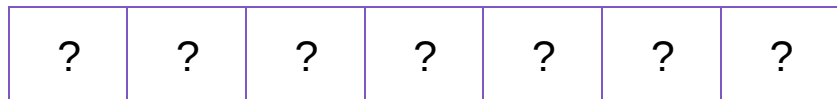
Duyệt hậu thứ tự

- Nếu Cây là rỗng Return
- Duyệt hậu thứ tự cây con gốc trái
- Duyệt hậu thứ tự cây con gốc phải
- Thăm A

2	5	11	6	7	4	9	5	2
---	---	----	---	---	---	---	---	---



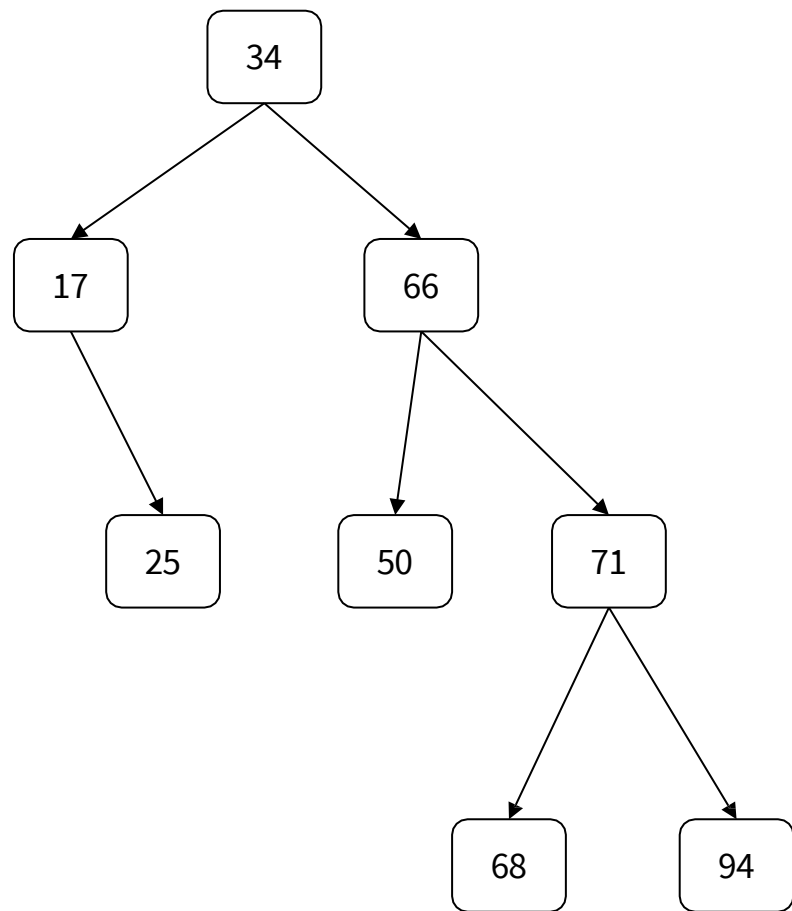
Duyệt hậu thứ tự



Cây nhị phân tìm kiếm

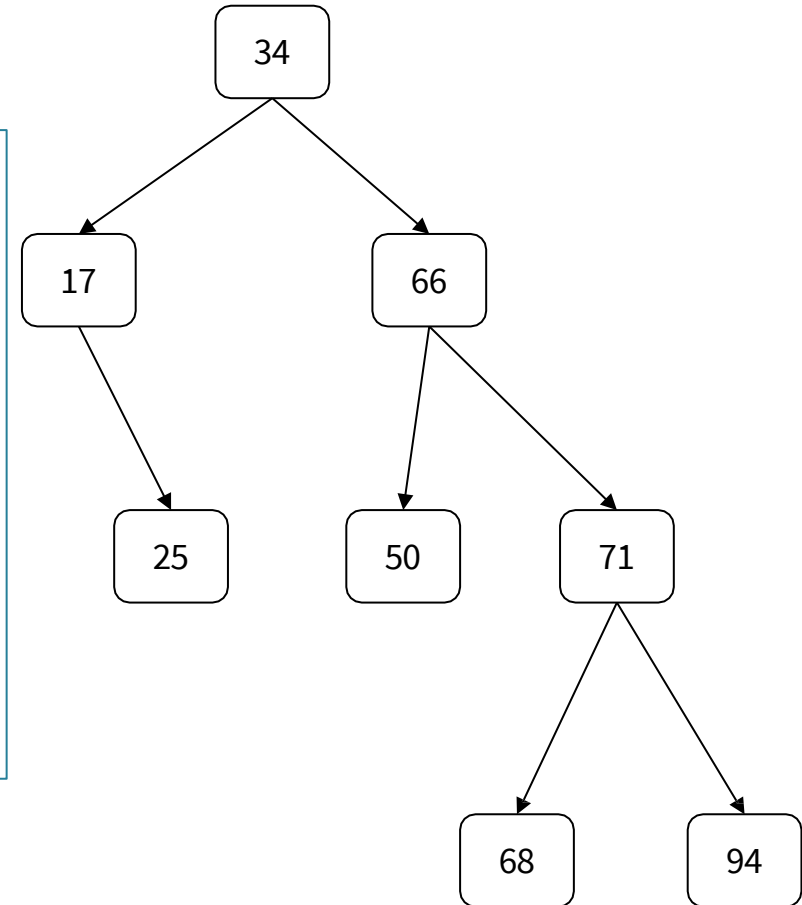
Cây nhị phân tìm kiếm (BST - Binary Search Tree) là cây nhị phân đều có tính chất sau:

Với mỗi nút x , các nút ở cây con bên trái của x có giá trị nhỏ hơn x , các nút ở cây con bên phải của x đều có giá trị lớn hơn hoặc bằng



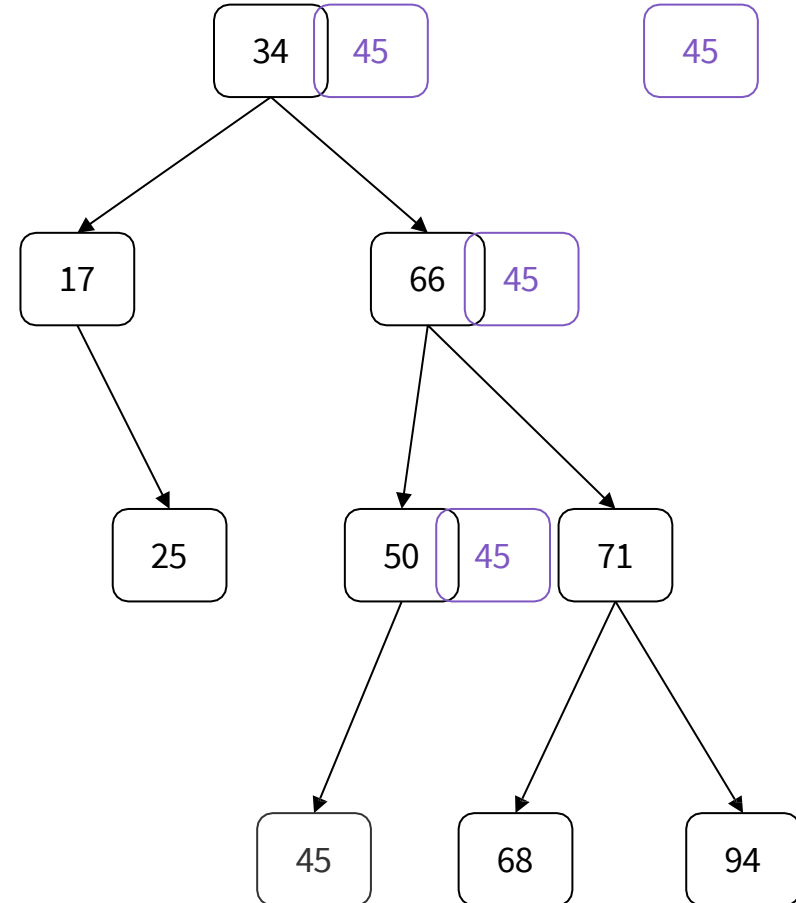
Cây nhị phân tìm kiếm

```
1  BST(node, key)
2    if node is Null then
3      return None
4    if key < node->value
5      return BST(node->left, key)
6    else if key > node->value
7      return BST(node->right, key)
8    else
9      return node
```



Xây dựng cây nhị phân tìm kiếm

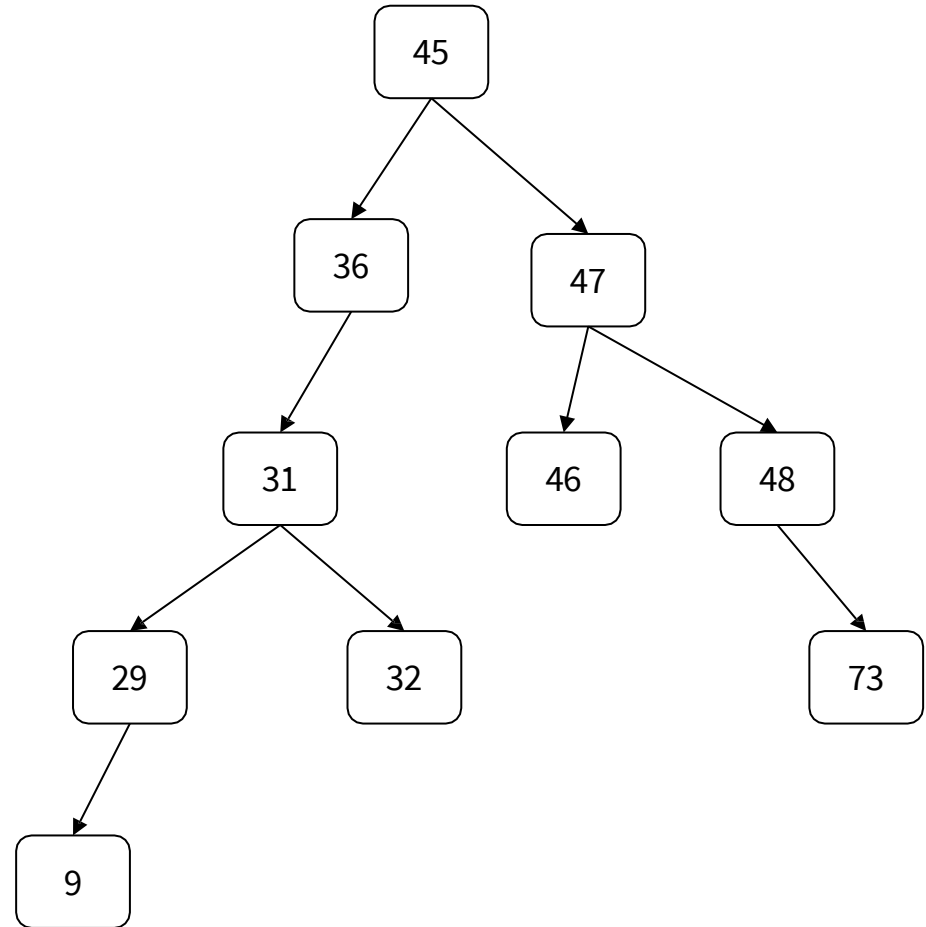
```
1 InsertNode(node treeN, node newN)
2   if (treeN == NULL)
3     treeN = newN
4   else if (newN->value < treeN->value)
5     InsertNode(treeN->left, newN)
6   else
7     InsertNode(treeN->right, newN)
```



Xây dựng cây nhị phân tìm kiếm

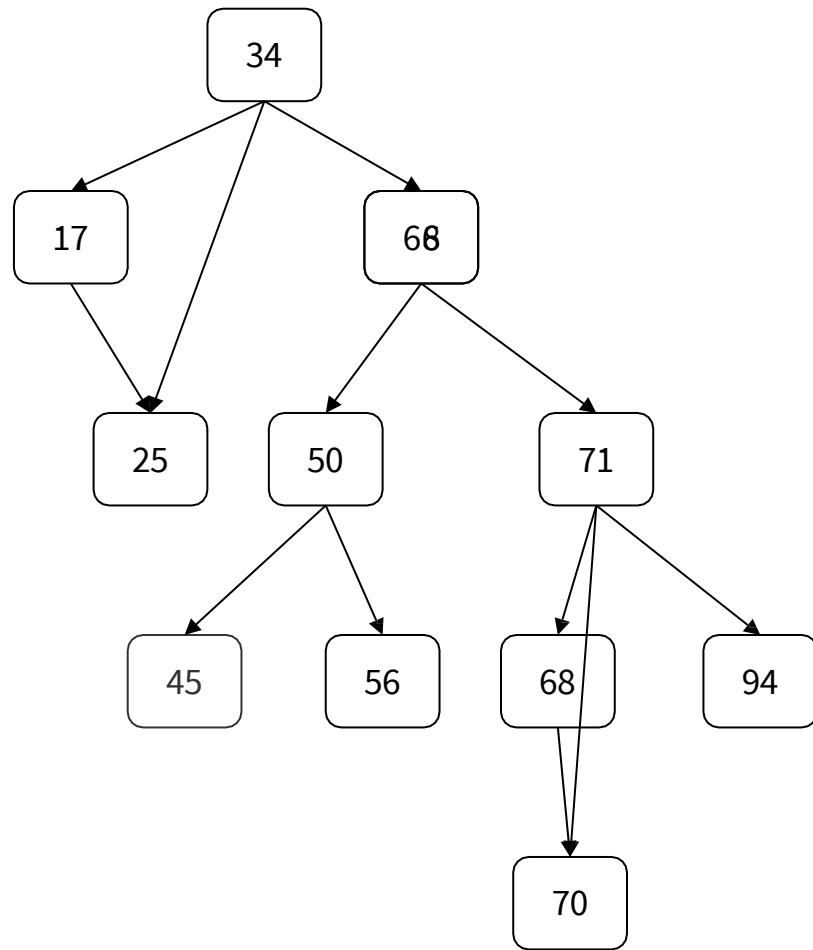
34, 47, 50, 11, 87, 85, 91, 71, 32

45, 47, 36, 31, 29, 48, 9, 73, 32, 46



Xây dựng cây nhị phân tìm kiếm

- Xoá nút lá
- Xoá nút có một con
- Xoá nút có hai con
 - Nút trái nhất của nhánh bên phải
 - Nút phải nhất của nhánh bên trái



```
1  DeleteNode (node nodeD)
2      if (nodeD->left == NULL)
3          nodeD = nodeD->right
4      else if (nodeD->right == NULL)
5          nodeD = nodeD->left
6      else
7          node temp = nodeD->right
8          while (temp->left != NULL)
9              temp = temp->left
10         nodeD->value = temp->value
11         DeleteNode (temp)
```

So sánh độ phức tạp

Tìm kiếm tuần tự: $O(n)$

Cây nhị phân tìm kiếm:

Xây dựng cây: $O(n \cdot \log_n)$

Tìm kiếm: $O(\log_n)$